

NETTOPOLOGY SUITE

Complete Implementation Guide

For the RePlot Land Readjustment Application

Application: RePlot Land Readjustment Tool

Framework: .NET 10, WinForms, SQLite, GDI+

NTS Version: 2.6.0 (already in your project)

Coordinate System: Nepal TM (EPSG:6207)

Every code example uses your actual classes: PointD, IShape, PolylineShape, ShapeManager, DrawingEngine, LandOwner, OriginalLandParcel, DatabaseHelper, LandOwnerRepository

Table of Contents

- Chapter 1: What NTS Actually Does (No Jargon)
- Chapter 2: Your Coordinate Types vs NTS Types (The Bridge)
- Chapter 3: GeometryFactory - The One Object You Create First
- Chapter 4: Core Geometry Classes - Complete Reference
- Chapter 5: Spatial Relationships - Asking Questions About Parcels
- Chapter 6: Geometric Operations - Doing Things To Parcels
- Chapter 7: Validation - Catching Bad Geometry
- Chapter 8: Spatial Indexing - Why Your R-Tree Matters
- Chapter 9: Reading and Writing Geometry (WKT, WKB, GeoJSON)
- Chapter 10: Database Integration With Your SQLite + Raw SQL
- Chapter 11: The Parcel Domain Entity - Connecting NTS to Your Models
- Chapter 12: Service Layer - GeometryService Implementation
- Chapter 13: Canvas Integration - Drawing NTS Geometry on Screen
- Chapter 14: Complete Workflows (Create, Merge, Split, Road Subtraction)
- Chapter 15: Performance Optimization
- Chapter 16: Common Mistakes and How to Avoid Them
- Chapter 17: Implementation Roadmap (What To Build In What Order)

Chapter 1: What NTS Actually Does

1.1 The Problem You Have Right Now

In your current RePlot app, a land parcel is a PolylineShape with IsClosed = true. It has vertices (corners), it can be drawn on screen, and it has a bounding box for the R-Tree spatial index.

But here is what it CANNOT do right now:

What You Need	Current Status	With NTS
Calculate parcel area in square meters	Not possible	<code>polygon.Area</code> (one line)
Check if two parcels touch each other	Not possible	<code>parcel1.Touches(parcel2)</code>
Check if a parcel overlaps a road	Not possible	<code>parcel.Intersects(road)</code>
Split a parcel with a line	Not possible	<code>parcel.Difference(blade)</code>
Merge two adjacent parcels	Not possible	<code>parcel1.Union(parcel2)</code>
Validate boundary (self-intersecting?)	Not possible	<code>polygon.IsValid</code>
Store geometry for QGIS/AutoCAD	Not possible	WKB binary format
Find all parcels near a point	R-Tree bbox only	R-Tree + precise geometry check

1.2 What NTS Is In Plain Language

NTS is a geometry math library. You give it coordinates, it gives you back answers. Think of it as the difference between a dumb list of numbers and an intelligent shape that understands itself.

Your PointD struct: Stores X and Y. That is all it does.

NTS Coordinate: Stores X, Y, Z. That is all it does.

Your PolylineShape: Stores a list of PointD. Can draw itself. Has a bounding box.

NTS Polygon: Stores coordinates AND understands area, perimeter, containment, intersection, union, difference, validation, centroid, buffer, simplification, and dozens more spatial operations.

NTS does NOT replace your drawing shapes. Your PolylineShape still draws on screen with GDI+. NTS lives behind the scenes doing the math. You convert between them when needed.

1.3 The Real-World Analogy

Imagine a surveyor with paper and pencil vs a surveyor with a Total Station instrument:

Paper Surveyor (Your PolylineShape)	Total Station (NTS Polygon)
Draws boundary lines	Draws boundary lines AND calculates area
Eyeballs if two plots touch	Mathematically proves adjacency
Manually traces road area	Computes exact road polygon from centerline
Cannot split a plot precisely	Splits to sub-millimeter accuracy
Cannot merge plots	Dissolves shared boundary automatically
Manual measurement error	IEEE double precision (15 significant digits)

1.4 Industry Standard

NTS implements the OGC Simple Features Specification. This is the same math standard used by AutoCAD Civil 3D, ESRI ArcGIS, QGIS, PostGIS, SQL Server Spatial, and Oracle Spatial. Data you create with NTS can be exported to any professional GIS or CAD software.

You already have NTS 2.6.0 installed in your project (check your .csproj). You also have ProjNET 2.1.0 for coordinate system transformations. Both are ready to use.

Chapter 2: Your Types vs NTS Types (The Bridge)

This is the most important chapter for understanding how NTS fits into your existing code. Your app has its own coordinate types. NTS has its own. Here is exactly how they map to each other.

2.1 Type Mapping

Your RePlot Type	NTS Equivalent	Key Difference
PointD (struct)	Coordinate (class)	NTS Coordinate has optional Z (elevation)
RectangleD (struct)	Envelope (class)	Envelope has Centre, Intersection, methods
PolylineShape (IsClosed=false)	LineString	NTS LineString has Length, Buffer, etc.
PolylineShape (IsClosed=true)	Polygon	NTS Polygon has Area, Contains, Union, etc.
RectangleShape	Polygon	NTS treats rectangles as 4-vertex polygons
CircleShape	Polygon (buffered point)	NTS approximates circle as 32+ vertex polygon
LineShape	LineString (2 points)	NTS LineString can have 2+ points
List<IShape>	GeometryCollection	NTS collection has Union(), Envelope, etc.

2.2 Converting Between Your Types and NTS

You will need two conversion directions. Here is the exact code:

PointD to NTS Coordinate and back

```
// Your PointD → NTS Coordinate
var coord = new Coordinate(myPointD.X, myPointD.Y);

// NTS Coordinate → Your PointD
var pointD = new PointD(coord.X, coord.Y);
```

RectangleD to NTS Envelope and back

```
// Your RectangleD → NTS Envelope
var envelope = new Envelope(
    rect.X,           // MinX
    rect.X + rect.Width, // MaxX
    rect.Y,           // MinY
    rect.Y + rect.Height // MaxY
);

// NTS Envelope → Your RectangleD
var rectD = new RectangleD(
    envelope.MinX,
    envelope.MinY,
    envelope.Width,
    envelope.Height
);
```

PolylineShape (closed) to NTS Polygon and back

```
// Your closed PolylineShape → NTS Polygon
public static Polygon ToNtsPolygon(PolylineShape shape, GeometryFactory factory)
{
    // Get vertices as NTS Coordinates
    var coords = shape.Vertices
        .Select(v => new Coordinate(v.X, v.Y))
        .ToList();

    // NTS requires the ring to be CLOSED (first == last)
    if (!coords[0].Equals2D(coords[coords.Count - 1]))
        coords.Add(new Coordinate(coords[0].X, coords[0].Y));

    var ring = factory.CreateLinearRing(coords.ToArray());
    return factory.CreatePolygon(ring);
}

// NTS Polygon → Your closed PolylineShape
public static PolylineShape ToPolylineShape(Polygon polygon)
{
    var vertices = polygon.ExteriorRing.Coordinates
        .Select(c => new PointD(c.X, c.Y))
        .ToList();

    // Remove the closing duplicate (your PolylineShape handles closure with IsClosed flag)
    if (vertices.Count > 1 &&
        vertices[0].X == vertices[vertices.Count - 1].X &&
        vertices[0].Y == vertices[vertices.Count - 1].Y)
    {
        vertices.RemoveAt(vertices.Count - 1);
    }

    return new PolylineShape(vertices, isClosed: true);
}
```

Critical rule: NTS Polygon rings MUST be closed (first coordinate == last coordinate). Your PolylineShape uses an IsClosed boolean flag instead. Always add the closing coordinate when converting TO NTS, and remove it when converting FROM NTS.

Chapter 3: GeometryFactory - Create This First

Every NTS geometry object is created through a GeometryFactory. You create ONE factory and reuse it everywhere. It controls two critical settings: precision and coordinate system.

3.1 Creating Your Factory

```
using NetTopologySuite;
using NetTopologySuite.Geometries;

// Create ONCE, store as a field or singleton
var precision = new PrecisionModel(1000.0); // 3 decimal places = 1mm accuracy
var factory = new GeometryFactory(precision, 6207); // SRID 6207 = Nepal TM
```

3.2 What the Parameters Mean

PrecisionModel - How Accurate Are Your Coordinates?

The number you pass to PrecisionModel is a scale factor. Coordinates get rounded to 1/scale decimal places.

Scale Factor	Decimal Places	Accuracy	Use Case
1.0	0	1 meter	Rough zoning boundaries
10.0	1	10 centimeters	Urban planning
100.0	2	1 centimeter	Construction
1000.0	3	1 millimeter	Land surveying (USE THIS)
PrecisionModels.Floating	15-17 digits	Full double precision	When you need maximum accuracy

Recommendation for RePlot: Use 1000.0 (millimeter precision). Land survey data in Nepal is typically accurate to centimeters, so millimeter precision gives you headroom without floating-point noise.

SRID - What Coordinate System Are Your Numbers In?

SRID (Spatial Reference ID) tells NTS what the X and Y numbers mean. Your RePlot app works with Nepal Transverse Mercator coordinates where X and Y are in meters.

SRID	Name	Units	When To Use
6207	Nepal TM	Meters	Your default - all canvas coordinates
4326	WGS 84	Degrees (lat/lon)	GPS data, Google Maps import
32644	UTM Zone 44N	Meters	Western Nepal survey data
32645	UTM Zone 45N	Meters	Eastern Nepal survey data

Critical: SRID does NOT transform coordinates. It is just a label. If you have GPS data in WGS84 (degrees) and canvas data in Nepal TM (meters), you must transform coordinates using ProjNET before mixing them. NTS will happily calculate nonsense distances between degrees and meters without complaining.

3.3 Where To Store the Factory in Your Architecture

```
// Option A: Static helper class (simplest for your current architecture)
public static class NtsFactory
{
    private static readonly GeometryFactory _instance =
        new GeometryFactory(new PrecisionModel(1000.0), 6207);

    public static GeometryFactory Instance => _instance;
}

// Usage anywhere in your code:
var polygon = NtsFactory.Instance.CreatePolygon(ring);

// Option B: Pass through constructor (when you add DI later)
public class GeometryService
{
    private readonly GeometryFactory _factory;

    public GeometryService()
    {
        _factory = new GeometryFactory(new PrecisionModel(1000.0), 6207);
    }
}
```

Chapter 4: Core Geometry Classes - Complete Reference

NTS has a hierarchy of geometry types. Here is every class you will use in RePlot, with what it represents in land readjustment work.

4.1 Coordinate - The Basic Building Block

A single X,Y point. Not a geometry object itself - just raw coordinate data.

Property/Method	Returns	What It Does
X	double	Easting in meters (Nepal TM)
Y	double	Northing in meters (Nepal TM)
Z	double	Elevation (NaN if not set)
Distance(other)	double	Straight-line distance to another Coordinate
Equals2D(other)	bool	True if X and Y match (ignores Z)

```
// Creating coordinates
var corner1 = new Coordinate(535123.45, 3012456.78);
var corner2 = new Coordinate(535200.00, 3012456.78);

// Distance between two coordinates (in meters, since we use Nepal TM)
double distance = corner1.Distance(corner2); // 76.55 meters
```

4.2 Point - A Coordinate with Geometry Powers

A Point wraps a single Coordinate but adds all spatial operations (distance, buffer, within, etc).

Use in RePlot: Survey benchmark locations, parcel centroids (for labeling), GPS waypoints, utility pole positions.

```
var point = factory.CreatePoint(new Coordinate(535123.45, 3012456.78));

// Properties
double x = point.X;           // 535123.45
double y = point.Y;           // 3012456.78

// Create a circle around the point (returns a Polygon with ~32 vertices)
```

```

var circle = point.Buffer(50.0); // 50-meter radius circle
double circleArea = circle.Area; // ~7853.98 sq meters

// Check if point is inside a parcel
bool isInside = point.Within(parcelPolygon);

```

4.3 LineString - Connected Sequence of Points

A series of coordinates connected by straight line segments. NOT closed.

Use in RePlot: Road centerlines, open boundaries, utility lines, measurement lines.

Property/Method	Returns	What It Does
Length	double	Total length of all segments in meters
NumPoints	int	Number of vertices
StartPoint	Point	First vertex
EndPoint	Point	Last vertex
IsClosed	bool	True if first == last coordinate
Centroid	Point	Geometric center
Buffer(distance)	Polygon	Creates polygon around line (like road surface)
Intersection(other)	Geometry	Where this line crosses another geometry

```

var roadCenterline = factory.CreateLineString(new[] {
    new Coordinate(535100, 3012400),
    new Coordinate(535200, 3012400),
    new Coordinate(535200, 3012500)
});

double roadLength = roadCenterline.Length; // 200 meters

// Create 20-meter wide road surface from centerline
Polygon roadSurface = (Polygon)roadCenterline.Buffer(10.0); // 10m each side

```

4.4 LinearRing - A Closed LineString (For Polygon Boundaries)

A LinearRing is a LineString where the first and last coordinates are identical. It MUST be closed. It MUST be simple (no self-crossings). You rarely create LinearRings directly - they are the building blocks of Polygons.

```
// CRITICAL: Last coordinate MUST equal first coordinate
var ring = factory.CreateLinearRing(new[] {
    new Coordinate(535100, 3012400),
    new Coordinate(535200, 3012400),
    new Coordinate(535200, 3012500),
    new Coordinate(535100, 3012500),
    new Coordinate(535100, 3012400) // MUST CLOSE - same as first!
});

bool valid = ring.IsValid;    // true if no self-intersections
bool closed = ring.IsClosed; // always true (by definition)
```

4.5 Polygon - THE Most Important Class for RePlot

A Polygon is an area bounded by one outer ring (shell) and zero or more inner rings (holes). This is your land parcel, your block boundary, your road surface, your building footprint.

Every parcel in your system will be an NTS Polygon.

Creating a Polygon

```
// Simple polygon (no holes) - a land parcel
var shell = factory.CreateLinearRing(new[] {
    new Coordinate(535100, 3012400),
    new Coordinate(535200, 3012400),
    new Coordinate(535200, 3012500),
    new Coordinate(535100, 3012500),
    new Coordinate(535100, 3012400) // close it
});
Polygon parcel = factory.CreatePolygon(shell);

// Polygon WITH holes (parcel with a pond or building footprint cut out)
var hole = factory.CreateLinearRing(new[] {
    new Coordinate(535130, 3012430),
    new Coordinate(535170, 3012430),
    new Coordinate(535170, 3012470),
    new Coordinate(535130, 3012470),
    new Coordinate(535130, 3012430)
});
Polygon parcelWithHole = factory.CreatePolygon(shell, new[] { hole });
```

Polygon Properties - Everything You Can Ask About a Parcel

Property	Returns	What It Tells You
----------	---------	-------------------

Area	double	Area in square meters (since Nepal TM is in meters)
Length	double	Perimeter in meters
Centroid	Point	Geometric center (may be outside for L-shaped parcels)
InteriorPoint	Point	GUARANTEED inside the polygon (use this for labels)
ExteriorRing	LinearRing	The outer boundary
NumInteriorRings	int	How many holes
GetInteriorRingN(i)	LinearRing	Get hole boundary by index
Coordinates	Coordinate[]	All vertices (outer + holes)
EnvelopeInternal	Envelope	Bounding box (for R-Tree)
IsValid	bool	No self-intersections, properly closed
IsEmpty	bool	Has no coordinates
IsSimple	bool	No self-crossings

```
// Using polygon properties for your parcel
double areaSqm = parcel.Area;           // 10000.0 (100m x 100m)
double perimeter = parcel.Length;       // 400.0 meters
Point center = parcel.InteriorPoint;    // guaranteed inside for label placement
Envelope bbox = parcel.EnvelopeInternal; // for R-Tree spatial index

// Convert area to Nepali units
double areaInRopani = areaSqm / 508.74;
double areaInBigha = areaSqm / 6772.63;
```

4.6 MultiPolygon - When One Owner Has Multiple Non-Adjacent Parcels

```
// Owner has 3 separate plots
Polygon plot1 = factory.CreatePolygon(ring1);
Polygon plot2 = factory.CreatePolygon(ring2);
Polygon plot3 = factory.CreatePolygon(ring3);
MultiPolygon estate = factory.CreateMultiPolygon(new[] { plot1, plot2, plot3 });

double totalArea = estate.Area;           // sum of all three
int plotCount = estate.NumGeometries;     // 3
Polygon first = (Polygon)estate.GetGeometryN(0);
```

4.7 Envelope - The Bounding Box (Critical for Performance)

Envelope is an axis-aligned rectangle. It is much faster to check than a full polygon. Your R-Tree spatial index stores Envelopes.

```
Envelope bbox = parcel.EnvelopeInternal;

double width = bbox.Width;    // extent in X direction
double height = bbox.Height;  // extent in Y direction
double minX = bbox.MinX;      // leftmost X
double maxY = bbox.MaxY;      // topmost Y
Coordinate center = bbox.Centre;

// Check if two bounding boxes overlap (FAST - no geometry math)
bool mightIntersect = bbox.Intersects(otherBbox);

// This is what your R-Tree does: fast bbox check first, then precise geometry check
var candidates = spatialIndex.Query(viewportEnvelope); // fast
var exact = candidates.Where(p => p.Geometry.Intersects(viewport)); // precise
```

Chapter 5: Spatial Relationships - Asking Questions About Parcels

These are the methods that make NTS invaluable for land readjustment. Every method returns true or false.

5.1 Relationship Methods

Method	What It Asks	RePlot Use Case
A.Intersects(B)	Do A and B share ANY point?	Find all parcels affected by a road
A.Touches(B)	Do A and B share boundary but NOT interior?	Find adjacent parcels (neighbors)
A.Overlaps(B)	Do A and B share area, but neither contains the other?	Detect ownership disputes
A.Contains(B)	Is B completely inside A?	Check if a point is inside a parcel
A.Within(B)	Is A completely inside B?	Inverse of Contains
A.Crosses(B)	Does a line cross through a polygon?	Does a road cross a parcel?
A.Disjoint(B)	Do A and B share NO points at all?	Confirm parcels are completely separate
A.Covers(B)	Like Contains but boundary contact counts	Check if parcel covers a point on its edge
A.Distance(B)	Shortest distance between A and B	Find parcels within 50m of a road
A.IsWithinDistance(B, d)	Is distance less than d? (optimized)	Fast proximity check

5.2 Practical Examples for Land Readjustment

Find all parcels that touch (are adjacent to) a selected parcel

```
// selectedParcel is an NTS Polygon
var adjacent = allParcels
    .Where(p => p.Id != selectedParcel.Id)
    .Where(p => p.Geometry.Touches(selectedParcel.Geometry))
    .ToList();
```

Find all parcels affected by a proposed road

```
// Create road polygon from centerline
LineString centerline = factory.CreateLineString(roadCoordinates);
Polygon roadSurface = (Polygon)centerline.Buffer(roadWidth / 2.0);

// Find every parcel the road touches or cuts through
var affectedParcels = allParcels
    .Where(p => p.Geometry.Intersects(roadSurface))
    .ToList();
```

Detect overlapping parcels (possible disputes)

```
var disputes = new List<(Parcel A, Parcel B, double OverlapArea)>();

for (int i = 0; i < parcels.Count; i++)
{
    for (int j = i + 1; j < parcels.Count; j++)
    {
        if (parcels[i].Geometry.Overlaps(parcels[j].Geometry))
        {
            var overlap = parcels[i].Geometry.Intersection(parcels[j].Geometry);
            disputes.Add((parcels[i], parcels[j], overlap.Area));
        }
    }
}
```

Check if a clicked point is inside any parcel

```
Point clickPoint = factory.CreatePoint(new Coordinate(worldX, worldY));

Parcel? clickedParcel = allParcels
    .FirstOrDefault(p => p.Geometry.Covers(clickPoint)); // Covers includes boundary
```


Chapter 6: Geometric Operations - Doing Things To Parcels

These methods create NEW geometry objects. NTS geometries are immutable - operations never modify the original.

6.1 Operations Reference

Operation	What It Does	RePlot Use Case
A.Union(B)	Merges A and B into one shape	Merge adjacent parcels (same owner)
A.Intersection(B)	Returns ONLY the area shared by both	Calculate how much road overlaps a parcel
A.Difference(B)	Returns A minus B (cuts B out of A)	Subtract road area from parcel
A.SymmetricDifference(B)	Returns everything NOT in the overlap	Rare - comparison visualization
A.Buffer(distance)	Expands (positive) or shrinks (negative) A	Setbacks, road surface from centerline
A.ConvexHull	Smallest convex polygon containing A	Simplified overview shape
A.Centroid	Geometric center point	Label placement (may be outside shape)
A.InteriorPoint	Point guaranteed inside A	Label placement (always inside shape)
A.Boundary	Returns the boundary as LineString	Extract just the outline
A.Simplify(tolerance)	Reduce vertex count	Performance at low zoom levels

6.2 Buffer - The Most Versatile Operation

Buffer creates a new polygon at a specified distance from the original. Positive expands, negative shrinks.

```
// Create road surface from centerline (10m each side = 20m wide road)
Polygon roadSurface = (Polygon)centerline.Buffer(10.0);

// Create building setback area (5m inward from parcel boundary)
Geometry buildableArea = parcel.Buffer(-5.0);
if (buildableArea is Polygon buildable)
    double netArea = buildable.Area; // area available for building
```

```
// Buffer with flat end caps (for roads that don't round at intersections)
var bufferParams = new BufferParameters {
    EndCapStyle = EndCapStyle.Flat,
    JoinStyle = JoinStyle.Round,
    QuadrantSegments = 16 // smoothness of curves
};
var roadFlat = BufferOp.Buffer(centerline, 10.0, bufferParams);
```

6.3 Union - Merging Parcels

```
// Merge two adjacent parcels
Polygon merged = (Polygon)parcel1.Union(parcel2);

// Merge many parcels at once (more efficient than pairwise)
var collection = factory.CreateGeometryCollection(parcel.ToArray());
Geometry allMerged = collection.Union();

// WARNING: If parcels don't touch, Union returns MultiPolygon, not Polygon
if (allMerged is Polygon single)
    Console.WriteLine("Merged into one polygon");
else if (allMerged is MultiPolygon multi)
    Console.WriteLine($"Result has {multi.NumGeometries} separate pieces");
```

6.4 Difference - Subtracting (Road from Parcel)

```
// Subtract road area from a parcel
Geometry remainder = parcel.Difference(roadSurface);

if (remainder is Polygon singlePiece)
{
    // Road only trimmed one side
    double lostArea = parcel.Area - singlePiece.Area;
}
else if (remainder is MultiPolygon pieces)
{
    // Road split the parcel into multiple pieces!
    for (int i = 0; i < pieces.NumGeometries; i++)
    {
        var piece = (Polygon)pieces.GetGeometryN(i);
        Console.WriteLine($"Piece {i}: {piece.Area:F2} sq.m");
    }
}
```

6.5 Intersection - Finding the Overlap

```
// How much of this parcel does the road take?
Geometry overlap = parcel.Intersection(roadSurface);
double areaLostToRoad = overlap.Area;

// What percentage of the parcel is taken?
double percentLost = (areaLostToRoad / parcel.Area) * 100;
```

6.6 Splitting a Parcel with a Line

```
public List<Polygon> SplitParcel(Polygon parcel, LineString splitLine)
{
    // Create a very thin polygon from the line (a "blade")
    var blade = splitLine.Buffer(0.001); // 1mm wide

    // Subtract the blade from the parcel
    var result = parcel.Difference(blade);

    var pieces = new List<Polygon>();
    if (result is Polygon single)
        pieces.Add(single);
    else if (result is MultiPolygon multi)
    {
        for (int i = 0; i < multi.NumGeometries; i++)
            pieces.Add((Polygon)multi.GetGeometryN(i));
    }
    else if (result is GeometryCollection gc)
    {
        for (int i = 0; i < gc.NumGeometries; i++)
            if (gc.GetGeometryN(i) is Polygon p)
                pieces.Add(p);
    }

    return pieces;
}
```

Chapter 7: Validation - Catching Bad Geometry

Invalid geometry causes silent calculation errors. A self-intersecting parcel returns wrong area. An unclosed ring throws exceptions. Always validate before saving.

7.1 Quick Validation

```
bool isValid = polygon.IsValid; // true or false
```

7.2 Detailed Validation (Find Where the Problem Is)

```
using NetTopologySuite.Operation.Valid;

var validator = new IsValidOp(polygon);
if (!validator.IsValid)
{
    var error = validator.ValidationErrors;
    string errorType = error.ErrorType.ToString();
    Coordinate location = error.Coordinate;

    Console.WriteLine($"Invalid: {errorType} at ({location.X}, {location.Y})");
    // Examples: "Self-intersection", "Ring not closed",
    //           "Hole lies outside shell", "Duplicate Points"
}
```

7.3 Fixing Invalid Geometry

```
// Method 1: Buffer(0) - fixes most topology errors automatically
Geometry fixed = invalidPolygon.Buffer(0);

// Method 2: GeometryFixer (NTS 2.x) - more robust
using NetTopologySuite.Geometries.Utilities;
var fixer = new GeometryFixer(invalidPolygon);
Geometry repaired = fixer.GetResult();
```

7.4 Common Validation Errors in Land Data

Error	Cause	Fix
-------	-------	-----

Self-intersection	Parcel boundary crosses itself (bowtie shape)	Buffer(0) or split into multiple parcels
Ring not closed	First coordinate does not equal last	Add closing coordinate
Duplicate Points	Same coordinate appears twice consecutively	Remove consecutive duplicates before creating geometry
Hole lies outside shell	Inner ring is not inside outer ring	Rebuild polygon or remove the hole
Too few points	Ring has fewer than 4 coordinates (3 unique + close)	Need at least 3 unique vertices for a valid polygon

7.5 Validation Helper for Your App

```

public static (bool IsValid, string Error, Coordinate? Location) Validate(Geometry
geom)
{
    if (geom == null)
        return (false, "Geometry is null", null);

    if (geom.IsEmpty)
        return (false, "Geometry is empty", null);

    var op = new IsValidOp(geom);
    if (op.IsValid)
        return (true, "", null);

    var err = op.ValidationErrorMessage;
    return (false, err.ErrorMessage, err.Coordinate);
}

```

Chapter 8: Spatial Indexing - Why Your R-Tree Matters

You already have an R-Tree spatial index using NTS STRtree. This chapter explains what it actually does and how to fix the performance bug in your current code.

8.1 The Problem Without an Index

Without a spatial index, finding which parcels are visible in the viewport requires checking EVERY parcel:

```
// WITHOUT index: O(n) - checks all 10,000 parcels every frame
var visible = allParcels.Where(p => p.BBox.IntersectsWith(viewport)).ToList();
```

With your R-Tree, the tree structure eliminates most parcels instantly:

```
// WITH R-Tree: O(log n) - checks ~50-100 candidates out of 10,000
var candidates = index.Query(viewportEnvelope); // returns only nearby shapes
```

8.2 The Bug In Your Current RTreeSpatialIndex

Your Insert() method calls Rebuild() on every single insert. When importing 5,000 shapes, this rebuilds the entire tree 5,000 times. This is $O(n^2)$ instead of $O(n \log n)$.

Here is your current code with the problem:

```
// CURRENT CODE (BROKEN - rebuilds on every insert)
public void Insert(IShape shape)
{
    _allShapes.Add(shape);
    Rebuild(); // <-- THIS rebuilds the entire tree!
}
```

8.3 The Fix - Dirty Flag Pattern

Mark the index as dirty when data changes. Only rebuild when a query actually needs results.

```
public class RTreeSpatialIndex
```

```

{
    private STRtree<IShape> _index;
    private Envelope _bounds;
    private List<IShape> _allShapes;
    private bool _isDirty = true; // NEW: dirty flag

    public RTreeSpatialIndex(RectangleD bounds)
    {
        _bounds = new Envelope(bounds.X, bounds.X + bounds.Width,
                                bounds.Y, bounds.Y + bounds.Height);
        _index = new STRtree<IShape>();
        _allShapes = new List<IShape>();
    }

    public void Insert(IShape shape)
    {
        _allShapes.Add(shape);
        _isDirty = true; // CHANGED: just mark dirty, don't rebuild
    }

    public bool Remove(IShape shape)
    {
        bool removed = _allShapes.Remove(shape);
        if (removed) _isDirty = true; // CHANGED: just mark dirty
        return removed;
    }

    public void Clear()
    {
        _allShapes.Clear();
        _isDirty = true;
    }

    public List<IShape> Query(RectangleD area)
    {
        if (_isDirty) // NEW: only rebuild when query needs it
        {
            Rebuild();
            _isDirty = false;
        }

        var env = ToEnvelope(area);
        var found = _index.Query(env);
        var result = new List<IShape>();
        foreach (var shape in found)
        {
            if (shape.GetBoundingBox().Intersects(area))
                result.Add(shape);
        }
        return result;
    }

    private void Rebuild()
    {
        _index = new STRtree<IShape>();
        foreach (var shape in _allShapes)
        {

```

```
        var env = ToEnvelope(shape.GetBoundingBox());
        _index.Insert(env, shape);
    }
    // Note: STRtree builds automatically on first query
}

public int GetTotalObjects() => _allShapes.Count;

private Envelope ToEnvelope(RectangleD rect)
{
    return new Envelope(rect.X, rect.X + rect.Width,
                        rect.Y, rect.Y + rect.Height);
}
}
```

Impact: Importing 5,000 shapes goes from 5,000 rebuilds to 1 rebuild. This is the single biggest performance fix you can make.

Chapter 9: Reading and Writing Geometry (WKT, WKB, GeoJSON)

NTS can serialize geometry to three formats. Each has a specific purpose.

9.1 Format Comparison

Format	Example Output	Size	Use Case
WKT (Well-Known Text)	POLYGON ((0 0, 100 0, 100 50, 0 50, 0 0))	Large (text)	Debugging, SQL queries, human reading
WKB (Well-Known Binary)	Binary bytes (not readable)	Small (50-70% less)	Database storage, file storage
GeoJSON	{"type":"Polygon","coordinates":[...]}	Medium (JSON)	Web maps, QGIS import/export, APIs

9.2 WKT - For Debugging and SQL

```
using NetTopologySuite.IO;

// Write geometry to human-readable text
var writer = new WKTWriter();
string wkt = writer.Write(parcel);
// Output: "POLYGON ((535100 3012400, 535200 3012400, ...))"

// Read geometry from text
var reader = new WKTRReader(NtsFactory.Instance);
Geometry geom = reader.Read("POLYGON ((0 0, 100 0, 100 50, 0 50, 0 0))");
```

9.3 WKB - For Your SQLite Database (USE THIS)

```
using NetTopologySuite.IO;

// Write to binary (for database BLOB column)
var wkbWriter = new WKBWriter();
byte[] bytes = wkbWriter.Write(parcel);

// Read from binary (when loading from database)
var wkbReader = new WKBReader(NtsFactory.Instance);
Geometry loaded = wkbReader.Read(bytes);
Polygon loadedParcel = (Polygon)loaded;
```

Why WKB over WKT for your database: WKB is 50-70% smaller, faster to parse, and has zero precision loss from text conversion. Your SQLite database stores it as a BLOB column.

9.4 GeoJSON - For Export to QGIS and Web Maps

```
using NetTopologySuite.IO;

// Write single geometry
var geoWriter = new GeoJsonWriter();
string json = geoWriter.Write(parcel);

// Read single geometry
var geoReader = new GeoJsonReader();
Geometry geom = geoReader.Read<Geometry>(json);

// Write a feature with properties (for QGIS import)
var feature = new NetTopologySuite.Features.Feature(parcel,
    new NetTopologySuite.Features.AttributesTable {
        { "ParcelNo", "123" },
        { "OwnerName", "Ram Bahadur" },
        { "AreaSqm", parcel.Area }
    });
string featureJson = geoWriter.Write(feature);
```

Chapter 10: Database Integration With Your SQLite

Your app uses raw SQL with System.Data.SQLite (not Entity Framework). This chapter shows exactly how to store and retrieve NTS geometry in your existing database pattern.

10.1 Add Geometry Column to Your Parcels Table

Add a BLOB column to tblOriginalLandParcels to store WKB geometry, plus bounding box columns for fast spatial filtering in SQL.

```
// In your DatabaseSchema.cs, add to CreateOriginalLandParcelsTable:
ALTER TABLE tblOriginalLandParcels ADD COLUMN GeometryWkb BLOB;
ALTER TABLE tblOriginalLandParcels ADD COLUMN BBoxMinX REAL;
ALTER TABLE tblOriginalLandParcels ADD COLUMN BBoxMinY REAL;
ALTER TABLE tblOriginalLandParcels ADD COLUMN BBoxMaxX REAL;
ALTER TABLE tblOriginalLandParcels ADD COLUMN BBoxMaxY REAL;
ALTER TABLE tblOriginalLandParcels ADD COLUMN GeometryAreaSqm REAL;
ALTER TABLE tblOriginalLandParcels ADD COLUMN GeometryPerimeter REAL;
```

Or better, add them in your schema creation SQL directly:

```
// Add these columns to your existing CREATE TABLE statement:
// GeometryWkb          BLOB,          -- NTS Polygon as WKB binary
// BBoxMinX              REAL,          -- Bounding box for spatial queries
// BBoxMinY              REAL,
// BBoxMaxX              REAL,
// BBoxMaxY              REAL,
// GeometryAreaSqm      REAL,          -- Cached area calculation
// GeometryPerimeter     REAL          -- Cached perimeter calculation
```

10.2 Saving Geometry to Database

// In your LandOwnerRepository.cs, modify InsertParcel or add new method:

```
public void SaveParcelGeometry(int parcelId, Polygon geometry)
{
    var wkbWriter = new WKBWriter();
    byte[] wkb = wkbWriter.Write(geometry);
    var bbox = geometry.EnvelopeInternal;

    const string sql = @"
        UPDATE tblOriginalLandParcels SET
            GeometryWkb = @Wkb,
```

```

        BBoxMinX = @MinX, BBoxMinY = @MinY,
        BBoxMaxX = @MaxX, BBoxMaxY = @MaxY,
        GeometryAreaSqm = @Area,
        GeometryPerimeter = @Perimeter
    WHERE ParcelId = @ParcelId";

    using var cmd = new SQLiteCommand(sql, _connection);
    cmd.Parameters.AddWithValue("@Wkb", wkb);
    cmd.Parameters.AddWithValue("@MinX", bbox.MinX);
    cmd.Parameters.AddWithValue("@MinY", bbox.MinY);
    cmd.Parameters.AddWithValue("@MaxX", bbox.MaxX);
    cmd.Parameters.AddWithValue("@MaxY", bbox.MaxY);
    cmd.Parameters.AddWithValue("@Area", geometry.Area);
    cmd.Parameters.AddWithValue("@Perimeter", geometry.Length);
    cmd.Parameters.AddWithValue("@ParcelId", parcelId);
    cmd.ExecuteNonQuery();
}

```

10.3 Loading Geometry from Database

```

public Polygon? LoadParcelGeometry(int parcelId)
{
    const string sql = "SELECT GeometryWkb FROM tblOriginalLandParcels WHERE ParcelId
= @Id";
    using var cmd = new SQLiteCommand(sql, _connection);
    cmd.Parameters.AddWithValue("@Id", parcelId);

    var result = cmd.ExecuteScalar();
    if (result == null || result == DBNull.Value)
        return null;

    byte[] wkb = (byte[])result;
    var wkbReader = new WKBReader(NtsFactory.Instance);
    return (Polygon)wkbReader.Read(wkb);
}

```

10.4 Spatial Query in SQL (Using Bounding Box Columns)

```

// Find all parcels visible in a viewport (fast SQL using bbox columns)
public List<int> QueryParcelsInViewPort(Envelope viewport)
{
    const string sql = @"
        SELECT ParcelId FROM tblOriginalLandParcels
        WHERE GeometryWkb IS NOT NULL
            AND BBoxMaxX >= @MinX AND BBoxMinX <= @MaxX
            AND BBoxMaxY >= @MinY AND BBoxMinY <= @MaxY";

    using var cmd = new SQLiteCommand(sql, _connection);

```

```
cmd.Parameters.AddWithValue("@MinX", viewport.MinX);
cmd.Parameters.AddWithValue("@MaxX", viewport.MaxX);
cmd.Parameters.AddWithValue("@MinY", viewport.MinY);
cmd.Parameters.AddWithValue("@MaxY", viewport.MaxY);

var ids = new List<int>();
using var reader = cmd.ExecuteReader();
while (reader.Read())
    ids.Add(reader.GetInt32(0));

return ids;
}

// For precise results, load the geometry and do exact intersection check:
public List<Polygon> QueryParcelsExact(Envelope viewport)
{
    var ids = QueryParcelsInViewports(viewport); // fast SQL filter
    var viewportPoly = NtsFactory.Instance.ToGeometry(viewport);

    var results = new List<Polygon>();
    foreach (var id in ids)
    {
        var geom = LoadParcelGeometry(id);
        if (geom != null && geom.Intersects(viewportPoly)) // precise check
            results.Add(geom);
    }
    return results;
}
```

Chapter 11: The Parcel Domain Entity

Right now your `OriginalLandParcel` model has `AreaInSqm` as a manually-entered number from Excel import. It has no geometry. This chapter shows how to connect your domain model to NTS.

11.1 Current vs Target State

Aspect	Current (OriginalLandParcel)	Target (with NTS)
Area	Manually typed number from Excel	Calculated from geometry (polygon.Area)
Boundary	Not stored	NTS Polygon stored as WKB in database
Adjacency	Unknown	Computed: <code>parcel.Touches(neighbor)</code>
Road contribution	Manual calculation	<code>parcel.Intersection(road).Area</code>
Validation	None	<code>polygon.IsValid</code> checks self-intersection
Export to QGIS	Not possible	GeoJSON with all properties

11.2 Updated OriginalLandParcel Model

Add geometry support to your existing model **WITHOUT** breaking current functionality:

```
// In your Models/LandOwnerModels.cs - ADD these properties to OriginalLandParcel
```

```
public class OriginalLandParcel
{
    // ... ALL your existing properties stay exactly as they are ...
    public int ParcelId { get; set; }
    public int LandOwnerId { get; set; }
    public string ParcelNo { get; set; } = string.Empty;
    // ... Province, District, etc. - keep everything ...
    public double? AreaInSqm { get; set; }
    // ... rest of existing properties ...

    // — NEW: Geometry properties —————
    /// <summary>
    /// NTS Polygon representing the parcel boundary.
    /// Null until geometry is imported from DXF or drawn on canvas.
    /// </summary>
    [System.ComponentModel.DataAnnotations.Schema.NotMapped]
    public Polygon? Geometry { get; set; }

    /// <summary>
```

```

    /// Area calculated from geometry (more accurate than manually entered AreaInSqm)
    /// </summary>
    public double? GeometryAreaSqm => Geometry?.Area;

    /// <summary>
    /// Perimeter calculated from geometry
    /// </summary>
    public double? GeometryPerimeter => Geometry?.Length;

    /// <summary>
    /// Center point for label placement (guaranteed inside the polygon)
    /// </summary>
    public PointD? LabelPosition
    {
        get
        {
            if (Geometry == null) return null;
            var pt = Geometry.InteriorPoint;
            return new PointD(pt.X, pt.Y);
        }
    }

    /// <summary>
    /// Bounding box for spatial queries
    /// </summary>
    public RectangleD? BoundingBox
    {
        get
        {
            if (Geometry == null) return null;
            var env = Geometry.EnvelopeInternal;
            return new RectangleD(env.MinX, env.MinY, env.Width, env.Height);
        }
    }

    // Navigation property (already exists)
    public LandOwner? Owner { get; set; }
}

```

Key point: The existing AreaInSqm from Excel import stays. GeometryAreaSqm is computed from geometry. When geometry is available, GeometryAreaSqm is the accurate one. When it is not (newly imported records with no DXF yet), AreaInSqm is the fallback.

Chapter 12: Service Layer - GeometryService

This is the central service that all other parts of your app call for geometry operations. It wraps NTS so the rest of your code never touches NTS directly.

12.1 Complete GeometryService Implementation

```
using NetTopologySuite.Geometries;
using NetTopologySuite.IO;
using NetTopologySuite.Operation.Valid;
using NetTopologySuite.Simplify;

namespace Land_Readjustment_Tool.Services
{
    public class GeometryService
    {
        private readonly GeometryFactory _factory;
        private readonly WKBWriter _wkbWriter;
        private readonly WKBReader _wkbReader;

        public GeometryService()
        {
            var precision = new PrecisionModel(1000.0);
            _factory = new GeometryFactory(precision, 6207);
            _wkbWriter = new WKBWriter();
            _wkbReader = new WKBReader(_factory);
        }

        public GeometryFactory Factory => _factory;

        // — Creation —————

        public Polygon CreatePolygon(Coordinate[] coordinates)
        {
            var closed = EnsureClosed(coordinates);
            var ring = _factory.CreateLinearRing(closed);
            return _factory.CreatePolygon(ring);
        }

        public Polygon CreatePolygonFromPointDs(List<PointD> vertices)
        {
            var coords = vertices
                .Select(v => new Coordinate(v.X, v.Y))
                .ToArray();
            return CreatePolygon(coords);
        }

        public LineString CreateLineString(Coordinate[] coordinates)
        {
            return _factory.CreateLineString(coordinates);
        }
    }
}
```



```

public Point CreatePoint(double x, double y)
{
    return _factory.CreatePoint(new Coordinate(x, y));
}

// — Conversion (your types <-> NTS) —————

public Polygon PolylineToPolygon(PolylineShape shape)
{
    return CreatePolygonFromPointDs(shape.Vertices);
}

public PolylineShape PolygonToPolyline(Polygon polygon)
{
    var vertices = polygon.ExteriorRing.Coordinates
        .Select(c => new PointD(c.X, c.Y))
        .ToList();

    // Remove closing duplicate
    if (vertices.Count > 1 &&
        vertices[0].X == vertices[^1].X &&
        vertices[0].Y == vertices[^1].Y)
        vertices.RemoveAt(vertices.Count - 1);

    return new PolylineShape(vertices, isClosed: true);
}

// — Validation —————

public (bool IsValid, string Error, Coordinate? Location)
    Validate(Geometry geometry)
{
    if (geometry == null) return (false, "Null", null);
    if (geometry.IsEmpty) return (false, "Empty", null);

    var op = new IsValidOp(geometry);
    if (op.IsValid) return (true, "", null);

    var err = op.ValidationErrorMessage;
    return (false, err, err.Coordinate);
}

public Geometry Fix(Geometry geometry)
{
    return geometry.IsValid ? geometry : geometry.Buffer(0);
}

// — Measurement —————

public double Area(Polygon polygon) => polygon.Area;
public double Perimeter(Polygon polygon) => polygon.Length;
public double Distance(Geometry a, Geometry b) => a.Distance(b);

public PointD GetLabelPoint(Polygon polygon)
{

```

```

        var pt = polygon.InteriorPoint;
        return new PointD(pt.X, pt.Y);
    }

    // — Operations —————

    public Polygon Merge(Polygon a, Polygon b)
    {
        var result = a.Union(b);
        if (result is Polygon p) return p;
        throw new InvalidOperationException(
            "Parcels are not adjacent - merge produced multiple polygons");
    }

    public List<Polygon> Split(Polygon parcel, LineString splitLine)
    {
        var blade = splitLine.Buffer(0.001);
        var diff = parcel.Difference(blade);

        var pieces = new List<Polygon>();
        if (diff is Polygon single) pieces.Add(single);
        else if (diff is MultiPolygon mp)
            for (int i = 0; i < mp.NumGeometries; i++)
                pieces.Add((Polygon)mp.GetGeometryN(i));
        else if (diff is GeometryCollection gc)
            for (int i = 0; i < gc.NumGeometries; i++)
                if (gc.GetGeometryN(i) is Polygon pg)
                    pieces.Add(pg);
        return pieces;
    }

    public Polygon SubtractRoad(Polygon parcel, Polygon roadSurface)
    {
        var result = parcel.Difference(roadSurface);
        if (result is Polygon p) return p;
        if (result is MultiPolygon mp)
            return (Polygon)mp.Geometries.OrderByDescending(g => g.Area).First();
        throw new InvalidOperationException("Road subtraction produced invalid
result");
    }

    public Polygon CreateRoadSurface(LineString centerline, double width)
    {
        return (Polygon)centerline.Buffer(width / 2.0);
    }

    public Polygon ApplySetback(Polygon parcel, double distance)
    {
        var result = parcel.Buffer(-distance);
        if (result is Polygon p) return p;
        if (result.IsEmpty)
            throw new InvalidOperationException("Setback too large - no area
remaining");
        throw new InvalidOperationException("Setback produced invalid geometry");
    }

    // — Serialization (for database) —————

```

```

public byte[] ToWkb(Geometry geometry) => _wkbWriter.Write(geometry);
public Geometry FromWkb(byte[] wkb) => _wkbReader.Read(wkb);

// — Simplification (for LOD rendering) —————

public Polygon Simplify(Polygon polygon, double tolerance)
{
    var result = DouglasPeuckerSimplifier.Simplify(polygon, tolerance);
    return result is Polygon p ? p : polygon;
}

// — Helpers —————

private Coordinate[] EnsureClosed(Coordinate[] coords)
{
    if (coords.Length < 3) throw new ArgumentException("Need at least 3
coordinates");
    if (coords[0].Equals2D(coords[^1])) return coords;

    var closed = new Coordinate[coords.Length + 1];
    Array.Copy(coords, closed, coords.Length);
    closed[^1] = new Coordinate(coords[0].X, coords[0].Y);
    return closed;
}
}

```

Chapter 13: Canvas Integration - Drawing NTS Geometry

NTS geometry must be converted to screen coordinates for GDI+ rendering. This happens through your existing `DrawingEngine.WorldToScreen` method. NTS stays in world coordinates, GDI+ works in screen pixels.

13.1 The Rendering Pipeline

Database (WKB) -> NTS Polygon (world meters) -> PolylineShape (world meters) -> WorldToScreen (pixels) -> GDI+ DrawPolygon (screen)

NTS is ONLY involved in steps 1 and 2. It never touches GDI+ or screen coordinates.

13.2 Rendering a Parcel from NTS Geometry

```
// In your rendering code, convert NTS Polygon to screen points:
public void RenderParcel(Graphics g, Polygon parcelGeometry,
    Func<PointD, PointD> worldToScreen, bool isSelected)
{
    var coords = parcelGeometry.ExteriorRing.Coordinates;

    // Convert world coordinates to screen coordinates
    var screenPoints = new PointF[coords.Length];
    for (int i = 0; i < coords.Length; i++)
    {
        var screen = worldToScreen(new PointD(coords[i].X, coords[i].Y));
        screenPoints[i] = new PointF(
            (float)Math.Round(screen.X),
            (float)Math.Round(screen.Y));
    }

    // Fill
    using var brush = new SolidBrush(
        isSelected ? Color.FromArgb(80, 0, 120, 215)
            : Color.FromArgb(30, 200, 200, 200));
    g.FillPolygon(brush, screenPoints);

    // Border
    using var pen = new Pen(
        isSelected ? Color.Yellow : Color.White,
        isSelected ? 2f : 0.25f);
    g.DrawPolygon(pen, screenPoints);

    // Label (area) at interior point
    var labelWorld = parcelGeometry.InteriorPoint;
    var labelScreen = worldToScreen(new PointD(labelWorld.X, labelWorld.Y));
```

```
string label = $"{parcelGeometry.Area:N0} m2";
using var font = new Font("Arial", 9);
using var textBrush = new SolidBrush(Color.White);
var size = g.MeasureString(label, font);
g.DrawString(label, font, textBrush,
    (float)labelScreen.X - size.Width / 2,
    (float)labelScreen.Y - size.Height / 2);
}
```

Performance note: In production, pool the Pen, Brush, and Font objects instead of creating them per-parcel. Your CanvasRenderer already pools grid pens - do the same for parcel rendering.

Chapter 14: Complete Workflows

These are end-to-end workflows that combine everything from previous chapters.

14.1 Workflow: Import DXF Parcels and Store Geometry

```
// When importing DXF, convert closed polylines to NTS Polygons
public void ImportDxfWithGeometry(string dxfPath, GeometryService geoService)
{
    var dxf = DxfDocument.Load(dxfPath);

    foreach (var entity in dxf.Entities.All)
    {
        if (entity.Type == EntityType.LwPolyline)
        {
            var lwPoly = (LwPolyline)entity;
            if (!lwPoly.IsClosed) continue; // skip open polylines

            var coords = lwPoly.Vertexes
                .Select(v => new Coordinate(v.Position.X, v.Position.Y))
                .ToArray();

            // Create NTS Polygon
            Polygon polygon = geoService.CreatePolygon(coords);

            // Validate
            var (isValid, error, loc) = geoService.Validate(polygon);
            if (!isValid)
            {
                polygon = (Polygon)geoService.Fix(polygon); // auto-fix
            }

            // Store as PolylineShape for canvas display
            PolylineShape displayShape = geoService.PolygonToPolyline(polygon);

            // Save geometry to database
            // (after matching to existing parcel by ParcelNo or spatial overlap)
            byte[] wkb = geoService.ToWkb(polygon);
            repository.SaveParcelGeometry(matchedParcelId, polygon);
        }
    }
}
```

14.2 Workflow: Merge Two Parcels (Same Owner Buys Neighbor)

```
public OriginalLandParcel MergeParcels(int parcelId1, int parcelId2,
    GeometryService geoService, LandOwnerRepository repo)
```

```

{
    // Load geometries
    var geom1 = repo.LoadParcelGeometry(parcelId1);
    var geom2 = repo.LoadParcelGeometry(parcelId2);

    if (geom1 == null || geom2 == null)
        throw new InvalidOperationException("Both parcels must have geometry");

    // Verify adjacency
    if (!geom1.Touches(geom2) && !geom1.Intersects(geom2))
        throw new InvalidOperationException("Parcels are not adjacent");

    // Merge
    Polygon merged = geoService.Merge(geom1, geom2);

    // Create new parcel record
    var newParcel = new OriginalLandParcel
    {
        LandOwnerId = existingOwner.LandOwnerId,
        ParcelNo = $"MERGED-{parcelId1}-{parcelId2}",
        AreaInSqm = merged.Area,
        // ... copy other fields as needed ...
    };
    newParcel.Geometry = merged;

    // Save to database
    int newId = repo.InsertParcel(newParcel);
    repo.SaveParcelGeometry(newId, merged);

    // Optionally: mark old parcels as superseded
    return newParcel;
}

```

14.3 Workflow: Calculate Road Contributions for Land Pooling

```

public Dictionary<int, double> CalculateRoadContributions(
    LineString roadCenterline, double roadWidth,
    GeometryService geoService, LandOwnerRepository repo)
{
    // Create road polygon
    Polygon roadSurface = geoService.CreateRoadSurface(roadCenterline, roadWidth);
    var roadBbox = roadSurface.EnvelopeInternal;

    // Find candidate parcels using bounding box (fast SQL query)
    var candidateIds = repo.QueryParcelsInViewPort(roadBbox);

    // Calculate each parcel's contribution
    var contributions = new Dictionary<int, double>();

    foreach (var parcelId in candidateIds)
    {
        var parcelGeom = repo.LoadParcelGeometry(parcelId);
    }
}

```

```

        if (parcelGeom == null) continue;

        if (parcelGeom.Intersects(roadSurface))
        {
            var overlap = parcelGeom.Intersection(roadSurface);
            double areaLost = overlap.Area;
            double percentLost = (areaLost / parcelGeom.Area) * 100;

            contributions[parcelId] = areaLost;
        }
    }

    return contributions; // ParcelId -> area lost in sq.m
}

```

14.4 Workflow: Split Parcel Along a Drawn Line

```

public List<OriginalLandParcel> SplitParcel(int parcelId,
    List<PointD> splitLineVertices,
    GeometryService geoService, LandOwnerRepository repo)
{
    var parcelGeom = repo.LoadParcelGeometry(parcelId);
    if (parcelGeom == null)
        throw new InvalidOperationException("Parcel has no geometry");

    // Convert drawn line to NTS LineString
    var coords = splitLineVertices
        .Select(v => new Coordinate(v.X, v.Y))
        .ToArray();
    var splitLine = geoService.CreateLineString(coords);

    // Split
    List<Polygon> pieces = geoService.Split(parcelGeom, splitLine);

    if (pieces.Count < 2)
        throw new InvalidOperationException("Line did not split the parcel");

    // Create new parcel records for each piece
    var newParcels = new List<OriginalLandParcel>();
    var originalParcel = repo.GetParcelById(parcelId);

    for (int i = 0; i < pieces.Count; i++)
    {
        var newParcel = new OriginalLandParcel
        {
            LandOwnerId = originalParcel.LandOwnerId,
            ParcelNo = $"{originalParcel.ParcelNo}-{(char)('A' + i)}",
            MapSheetNo = originalParcel.MapSheetNo,
            AreaInSqm = pieces[i].Area,
            // copy other fields ...
        };
        newParcel.Geometry = pieces[i];
    }
}

```



```
        int newId = repo.InsertParcel(newParcel);
        repo.SaveParcelGeometry(newId, pieces[i]);
        newParcels.Add(newParcel);
    }

    return newParcels;
}
```

Chapter 15: Performance Optimization

15.1 Two-Stage Filtering (The Most Important Pattern)

Every spatial query in your app should follow this pattern: fast bounding-box filter first, then precise geometry check on the small set of candidates.

```
// STAGE 1: Fast bounding box (R-Tree or SQL) - eliminates 95% of shapes
var candidates = spatialIndex.Query(viewportEnvelope);

// STAGE 2: Precise geometry check - only on the ~5% of candidates
var exact = candidates
    .Where(c => c.Geometry.Intersects(viewportPolygon))
    .ToList();
```

15.2 Level-of-Detail for Zoom Levels

```
// At low zoom (zoomed out), simplify geometry to reduce vertex count
public Polygon GetParcelForZoom(Polygon original, double zoomScale)
{
    if (zoomScale > 1.0) return original;           // zoomed in: full detail
    if (zoomScale > 0.1) return Simplify(original, 0.5); // medium
    if (zoomScale > 0.01) return Simplify(original, 2.0); // low detail
    return (Polygon)original.ConvexHull;           // very far out: just the hull
}
```

15.3 Batch Loading Geometry

```
// Load all parcel geometries in one SQL query (not one query per parcel)
public Dictionary<int, Polygon> LoadAllGeometries()
{
    const string sql = "SELECT ParcelId, GeometryWkb FROM tblOriginalLandParcels WHERE GeometryWkb IS NOT NULL";
    var result = new Dictionary<int, Polygon>();

    using var cmd = new SQLiteCommand(sql, _connection);
    using var reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        int id = reader.GetInt32(0);
        byte[] wkb = (byte[])reader[1];
        result[id] = (Polygon)_wkbReader.Read(wkb);
    }
}
```

```

    return result;
}

```

15.4 Performance Numbers to Expect

Operation	10 Parcels	1,000 Parcels	10,000 Parcels
R-Tree query (viewport)	<1 ms	<1 ms	1-2 ms
Polygon.Area	<0.01 ms	N/A (per-parcel)	N/A (per-parcel)
Polygon.Intersects	<0.1 ms	N/A (per-pair)	N/A (per-pair)
Union (merge 2 parcels)	1-5 ms	N/A (per-pair)	N/A (per-pair)
Split (line through parcel)	2-10 ms	N/A (per-parcel)	N/A (per-parcel)
Buffer (road surface)	1-3 ms	N/A (per-line)	N/A (per-line)
Load all WKB from SQLite	<1 ms	50-100 ms	200-500 ms
Build R-Tree index	<1 ms	10-20 ms	50-100 ms

Chapter 16: Common Mistakes and How to Avoid Them

Mistake 1: Forgetting to Close the Ring

```
// WRONG - throws exception
var ring = factory.CreateLinearRing(new[] {
    new Coordinate(0, 0), new Coordinate(100, 0),
    new Coordinate(100, 100), new Coordinate(0, 100)
    // Missing: new Coordinate(0, 0) to close!
});

// CORRECT - always close (or use the helper in GeometryService)
var ring = factory.CreateLinearRing(new[] {
    new Coordinate(0, 0), new Coordinate(100, 0),
    new Coordinate(100, 100), new Coordinate(0, 100),
    new Coordinate(0, 0) // CLOSED
});
```

Mistake 2: Mixing Coordinate Systems

```
// WRONG - mixing GPS degrees with Nepal TM meters
var gpsPoint = factory.CreatePoint(new Coordinate(85.3240, 27.7172)); // degrees!
var tmParcel = LoadParcelFromDb(); // meters!
double dist = gpsPoint.Distance(tmParcel.Geometry); // NONSENSE result!

// CORRECT - transform GPS to Nepal TM first using ProjNET
// (You already have ProjNET 2.1.0 in your packages)
```

Mistake 3: Assuming Union Always Returns Polygon

```
// WRONG - crashes if parcels don't touch
Polygon merged = (Polygon)parcel1.Union(parcel2);

// CORRECT - handle both cases
var result = parcel1.Union(parcel2);
if (result is Polygon single)
    return single;
else if (result is MultiPolygon multi)
    throw new InvalidOperationException(
        $"Parcels are not adjacent - got {multi.NumGeometries} pieces");
```

Mistake 4: Not Updating Bounding Box After Geometry Changes

```
// WRONG - bbox in database is stale after modification
parcelGeom = (Polygon)parcelGeom.Buffer(-5.0);
repo.SaveParcelWkb(parcelId, wkbWriter.Write(parcelGeom));
// BBox columns still have OLD values! SQL spatial queries return wrong results!

// CORRECT - always update bbox when geometry changes
parcelGeom = (Polygon)parcelGeom.Buffer(-5.0);
repo.SaveParcelGeometry(parcelId, parcelGeom); // method updates bbox too
```

Mistake 5: Creating New GDI+ Pens Per Shape Per Frame

```
// WRONG - 10,000 shapes x 60 fps = 600,000 Pen allocations per second
foreach (var shape in shapes)
    using (var pen = new Pen(Color.White, 0.25f)) // GARBAGE PRESSURE
        g.DrawPolygon(pen, points);

// CORRECT - create once, reuse
private readonly Pen _parcelPen = new Pen(Color.White, 0.25f);

foreach (var shape in shapes)
    g.DrawPolygon(_parcelPen, points); // zero allocations
```

Chapter 17: Implementation Roadmap

Implement in this exact order. Each phase produces a working application. Do not skip phases.

Phase 1: Foundation (Day 1)

Goal: NTS factory exists, GeometryService works, R-Tree bug is fixed.

1. Create NtsFactory static class (Chapter 3)
2. Create GeometryService class (Chapter 12)
3. Fix RTreeSpatialIndex with dirty flag (Chapter 8)
4. Write a simple test: create a polygon, check its area, validate it

Phase 2: Database Storage (Day 2-3)

Goal: Parcel geometry persists to SQLite.

1. Add geometry columns to tblOriginalLandParcels (Chapter 10)
2. Add SaveParcelGeometry and LoadParcelGeometry to LandOwnerRepository (Chapter 10)
3. Add QueryParcelsInViewport SQL method (Chapter 10)
4. Test: save a polygon, reload it, verify area matches

Phase 3: DXF Import with Geometry (Day 4-5)

Goal: Imported DXF parcels have NTS geometry stored in database.

1. Modify DXF import to create NTS Polygons from closed polylines (Chapter 14.1)
2. Validate each polygon and auto-fix invalid ones (Chapter 7)
3. Match imported geometry to existing parcel records by ParcelNo or spatial overlap
4. Save geometry to database after import

Phase 4: Canvas Rendering from NTS (Day 6-7)

Goal: Parcels with geometry render on canvas with area labels.

1. Load parcel geometries when opening a project
2. Convert NTS Polygons to PolylineShapes for the ShapeManager
3. Render area labels at InteriorPoint positions (Chapter 13)
4. Color parcels by owner or land use

Phase 5: Spatial Operations (Day 8-10)

Goal: Merge, split, and road subtraction work.

1. Implement parcel merge (Chapter 14.2)
2. Implement parcel split (Chapter 14.4)
3. Implement road contribution calculation (Chapter 14.3)
4. Add adjacency detection (find neighbors)

Phase 6: Export (Day 11-12)

Goal: Parcels can be exported to GeoJSON for QGIS.

1. Implement GeoJSON export with parcel properties (Chapter 9.4)
2. Implement GeoJSON import
3. Test round-trip: export from RePlot, open in QGIS, re-import

Summary

NTS is already installed in your project. The hardest part - choosing the right library, setting up the architecture, building the coordinate system - is done. What remains is connecting the dots between your existing shapes (PolylineShape, ShapeManager, DrawingEngine) and the NTS geometry operations (Polygon, Union, Difference, Buffer).

The GeometryService in Chapter 12 is the bridge. Build that first, and everything else follows.

End of Document

RePlot Land Readjustment Tool - NetTopologySuite Implementation Guide